Advanced Quantitative Research Methods

Chapter 8: Tree-Based Methods

Prof. Dr. Christoph Weisser

Slides, lab notebooks and data:
github.com/ChrisW09/Advanced-Quantitative-Research-Methods

The course at a glance

Chapter	Topic
Ch. 1	Introduction
Ch. 2	Statistical learning — accuracy, bias–variance
Ch. 3	Linear regression — estimation, inference, diagnostics
Ch. 4	Classification — logistic regression, confusion matrix, ROC
Ch. 5	Resampling — validation, CV, bootstrap
Ch. 6	Model selection and regularisation — ridge, lasso
Ch. 7	Moving beyond linearity — splines, GAMs
Ch. 8	**Tree-based methods** — trees, forests, boosting

Contents

1. Why this chapter matters
2. Trees
3. Bagging / RF
4. Boosting
5. Python Lab
6. Summary

Optional and advanced material is collected in the appendix at the end of the deck.

Notation in this chapter

Symbol	Meaning
R_j	Rectangular region (leaf) of the partitioned predictor space
$\bar{y}_{R_j}$	Mean training response in region R_j (leaf prediction)
J, $\lvert T \rvert$	Number of leaves of a tree T (its complexity)
α	Cost-complexity pruning parameter: price paid per leaf
$\hat{p}_{mk}$	Fraction of node-m training points in class k
E	Error rate $1 - \max_k \hat{p}_{mk}$: node impurity
G	Gini index $\sum_k \hat{p}_{mk}(1 - \hat{p}_{mk})$: node impurity
D	Cross-entropy $-\sum_k \hat{p}_{mk} \log \hat{p}_{mk}$: node impurity
B	Number of bootstrap samples / trees in an ensemble
m	Predictors tried per split in a random forest ($m = p$: bagging)
ρ	Pairwise correlation between the ensemble's trees
λ (or ν), d	Boosting learning rate (shrinkage) and tree depth

Sources and references

Primary textbook

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning, with Applications in Python.* Springer.

About these slides

Each method is introduced with motivation, intuition, formal definition, and worked examples. Slide titles are descriptive.

Learning objectives for this chapter

By the end of this chapter you will be able to:

- **Build** regression and classification trees by recursive binary splitting; **apply** cost-complexity pruning.
- **Explain** why a single tree is high-variance and how *bagging* reduces that variance.
- **Distinguish** bagging from random forests; explain the benefit of feature subsampling at each split.
- **State** the boosting algorithm and its three tuning parameters.
- **Choose** between trees, random forests, and boosting for a given tabular problem.

How to use this deck — self-study and revision

Attempt each exercise *before* its solution; run `chapter_08_lab.ipynb` alongside the lab section. Revise from the back — key formulas, vocabulary, self-check (each question names its page) — then redo the exercises; the appendix is fully solved.

Roadmap of this chapter

From single trees to industrial ensembles

1. Single trees: regression and classification, pruning.
2. Bagging: variance reduction by averaging.
3. Random forests: bagging plus feature subsampling.
4. Boosting: sequential additive trees on residuals.
5. BART: Bayesian sum of trees.

Outline

1. Why this chapter matters
2. Trees
3. Bagging / RF
4. Boosting
5. Python Lab
6. Summary

Why trees dominate tabular data

Tree-based methods are the dominant non-linear method on tabular data.

Five reasons

- Capture nonlinearity *and* interactions automatically.
- Interpretable as a sequence of yes/no questions.
- Handle mixed predictor types without dummy coding.
- Robust to outliers and monotone transformations.
- Ensembles (random forests, gradient boosting) routinely win Kaggle competitions on tabular data.

Single trees are weak: ensembles fix it

The single-tree pathology

A single tree is high-variance, blocky, and unstable. Small data changes produce very different trees.

Takeaway

The fix is to build many trees and aggregate. Bagging, random forests, gradient boosting, and BART all do this — in different ways.

Outline

1 Why this chapter matters

2 Trees

3 Bagging / RF

4 Boosting

5 Python Lab

6 Summary

Regression trees: idea

A regression tree is a piecewise-constant predictor over a partition of the predictor space.

The two-step view

1. Partition the predictor space into J rectangular regions $R_1, \ldots, R_J$.
2. Predict the mean response $\bar{y}_{R_j}$ on each region.

Goal

Minimise within-region variability: $\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^2$.

Live demo — §2 of the notebook

Switch to `chapter_08_lab.ipynb` §2 and run the `plot_tree` cell: a depth-4 regression tree on the Carseats data, with `ShelveLoc` and `Price` taking the top splits and a test MSE of 3.612 printed by the next cell.

History: this is CART — Breiman, Friedman, Olshen & Stone (1984), the book that made trees a serious statistical tool.

Reading the tree-fit objective

How to read this

Symbol	Meaning
J	Number of leaves (regions)
R_j	Rectangular region in $\mathbb{R}^p$
$\bar{y}_{R_j}$	Mean of y-values inside region j

Takeaway

Finding the globally optimal partition is computationally infeasible. In practice we use *recursive binary splitting*.

Recursive binary splitting: one step

The greedy step

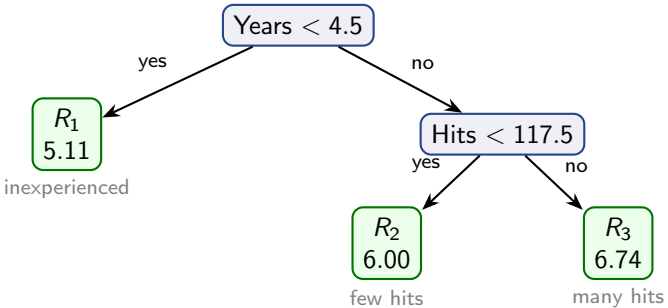
At each node:

1. For every predictor X_j and every cutpoint s, compute the RSS (residual sum of squares) that would result from splitting on $X_j < s$.
2. Pick the (j, s) that minimises the resulting RSS.
3. Recurse on each child.

Greedy growth and stopping rules

"Greedy" means each split optimises the *current* node only, never looking ahead — fast, but not the globally optimal tree (hence grow-then-prune). Stop on minimum leaf size, maximum depth, or minimum RSS gain.

The Hitters salary tree, drawn as a flowchart



Takeaway

Each split is a yes/no question; each leaf reports the mean log-salary there. The highest earners sit in the deep right branch: experienced players with many hits.

Why prune?

Recursive splitting with no stopping rule gives one leaf per training point — zero training error, pathological test error.

Cost-complexity pruning

Choose a tree T minimising:

$$\sum_{j=1}^{|T|} \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^2 + \alpha |T|.$$

How to read this

Here $|T|$ is the number of leaves and $\alpha \geq 0$ the price paid per leaf: the first term rewards training fit, the $\alpha|T|$ term punishes size. Larger $\alpha \Rightarrow$ smaller pruned tree; choose α by cross-validation.

Classification trees: splitting rules

Same recursive splitting; predict the majority class in each leaf. RSS no longer applies, so we score a node by its *impurity*:

Error rate, Gini and cross-entropy in node m

$$E = 1 - \max_k \hat{p}_{mk}, \quad G = \sum_k \hat{p}_{mk}(1 - \hat{p}_{mk}), \quad D = -\sum_k \hat{p}_{mk} \log \hat{p}_{mk}.$$

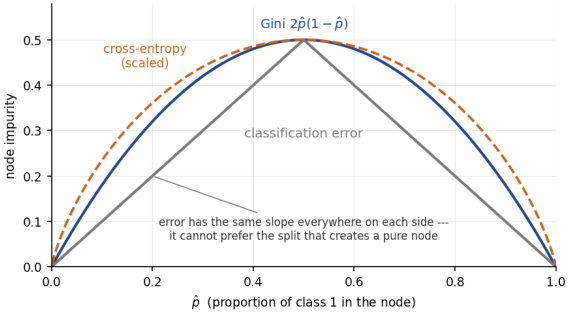
Reading the impurities, and scoring a split

$\hat{p}_{mk}$ is the fraction of node-m points in class k; E is the share the majority vote gets wrong. All three are *small* when one class dominates ($\hat{p}_{mk} \approx 1$) and *large* at a 50/50 mix. A *split* is scored by the **size-weighted** average of its two children, $\frac{n_L}{n} Q_L + \frac{n_R}{n} Q_R$ — a big impure child must count for more than a small one.

Why not the error rate?

E is blind to purity gains that do not flip the majority class, so it ties too often to grow a tree with. Split on Gini or cross-entropy; keep E for pruning and for reporting accuracy. *ISLP §8.1.2, (8.5)–(8.7)*

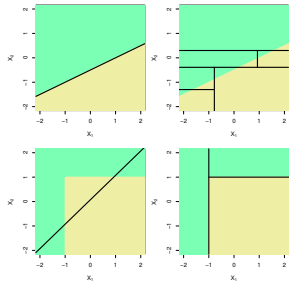
The three impurity measures, drawn



Takeaway

Gini and cross-entropy are strictly concave, so any split that moves a node toward purity lowers them. Classification error is piecewise *linear* — flat to purity gains that do not flip the majority — which is exactly what Exercise 8.2 exploits.

Trees vs. linear models: when each shines



Takeaway

Top: with a linear boundary the linear model wins and the tree's box-steps lose. Bottom: with non-linear, interacting structure the tree wins.

Source: Figure 8.7 — ISLP, James et al. (2023).

Strengths and weaknesses of single trees

Strengths

- Easy to explain.
- Handle mixed and missing data.
- Capture interactions naturally.
- Robust to monotone transformations.

Weaknesses

- Unstable: small data changes $\rightarrow$ very different trees.
- Predictions are step functions $\rightarrow$ rough.
- Often beaten by linear models on simple problems.

Takeaway

A small pruned tree survives precisely because it is weak but *printable* — a rule set you can read off one page. Grow it deep and you lose that without gaining much accuracy.

Exercise 8.1 — Regression tree split by RSS [Math]

Exercise (≈5 min) — try it before the solution

A regression tree is grown on this dataset:

x	1	2	3	4	5	6
y	1.0	1.5	2.0	8.0	8.5	9.0

1. List the candidate split points (midpoints between consecutive x).
2. Compute the total RSS for the splits $x < 3.5$ and $x < 2.5$.
3. Which split is chosen? Give the predictions for $x = 2$ and $x = 5$.
4. Predict y for a new point $x = 3.2$.

Solution — Exercise 8.1 (1/2)

Solution

(a) Candidate splits. Any cutpoint between two consecutive x-values produces the same partition, so the midpoints are the canonical candidates: $s \in \{1.5,\ 2.5,\ 3.5,\ 4.5,\ 5.5\}$.

(b) RSS per split. In each region the RSS-minimising constant prediction is the region *mean*; the RSS adds the squared deviations from it.
Split $x < 3.5$*:* left $\{1.0, 1.5, 2.0\}$, mean 1.5; right $\{8.0, 8.5, 9.0\}$, mean 8.5.

$$\mathrm{RSS}_L = (-0.5)^2 + 0^2 + 0.5^2 = 0.5, \quad \mathrm{RSS}_R = 0.5, \quad \text{total} = 1.0.$$

Split $x < 2.5$*:* left $\{1.0, 1.5\}$, mean 1.25; right $\{2.0, 8.0, 8.5, 9.0\}$, mean 6.875.

$$\mathrm{RSS}_L = (-0.25)^2 + 0.25^2 = 0.125,$$

$$\mathrm{RSS}_R = (-4.875)^2 + 1.125^2 + 1.625^2 + 2.125^2 \approx 32.19, \quad \text{total} \approx 32.31.$$

Solution — Exercise 8.1 (2/2)

Solution

(c) Chosen split and predictions. The split $x < 3.5$ gives total RSS 1.0 versus ≈ 32.31 for $x < 2.5$, so $x < 3.5$ is chosen — it separates the two tight y-clusters, whereas $x < 2.5$ leaves the outlying $y = 2.0$ stranded among the large values. Predictions are the leaf means: $x = 2$ lands left $\Rightarrow \hat{y} = 1.5$; $x = 5$ lands right $\Rightarrow \hat{y} = 8.5$.

(d) New point. $x = 3.2 < 3.5$, so it falls in the left region: $\hat{y} = 1.5$.

Interpretation: a regression tree predicts a *step function* — every point in a leaf gets the same value, no matter how close it sits to the boundary.

Common mistake

Interpolating between the two leaf means for a point near the cut (e.g. predicting ≈ 5 at $x = 3.2$). Trees do not interpolate; the prediction jumps only when x crosses 3.5.

Exercise 8.2 — Gini index vs. classification error [Math]

Exercise (≈5 min) — try it before the solution

A binary node contains 400 positive and 400 negative cases (800 total). Two candidate splits give:

- **Split 1:** region A $(300+, 100-)$ and region B $(100+, 300-)$.
- **Split 2:** region A $(200+, 400-)$ and region B $(200+, 0-)$.

1. Compute the weighted classification error for each split.
2. Compute the weighted Gini index for each split.
3. Which split does Gini prefer, and why is that desirable?
4. Why are Gini / entropy preferred for *growing* the tree?

Check it in code — §6 of the notebook

§6 of `chapter_08_lab.ipynb` recomputes this exercise: both splits print error 0.250, while Gini prints 0.375 vs. 0.333 and picks the split with the pure node.

Solution — Exercise 8.2 (1/4)

Solution

Set-up. For a node with positive proportion $\hat{p}$: error $E = 1 - \max(\hat{p}, 1-\hat{p})$ (the fraction misclassified by the majority vote) and Gini $G = 2\hat{p}(1-\hat{p})$. A split is scored by the *weighted* average of its two regions, each weighted by its share of the 800 cases — a large impure region must count more than a small one.

Solution — Exercise 8.2 (2/4)

Solution

(a) Classification error. *Split 1:* each region has 400 cases with $\hat{p} = 300/400 = 0.75$, so $E = 1 - 0.75 = 0.25$ each; weighted:

$$\tfrac{400}{800}(0.25) + \tfrac{400}{800}(0.25) = 0.25.$$

Split 2: region A (600 cases) is majority negative, $E = 200/600 = 0.333$; region B (200 cases, pure) has $E = 0$. Weighted:

$$\tfrac{600}{800}(0.333) + \tfrac{200}{800}(0) = 0.25.$$

Both splits give error 0.25 — the error criterion finds them *indistinguishable*.

Solution — Exercise 8.2 (3/4)

Solution

(b) Gini. *Split 1:* each region $\hat{p} = 0.75$, $G = 2(0.75)(0.25) = 0.375$; weighted $= 0.375$. *Split 2:* region A $\hat{p} = 200/600 = 1/3$, $G = 2(\frac{1}{3})(\frac{2}{3}) = 0.444$, weight 0.75; region B pure, $G = 0$. Weighted $= 0.75(0.444) + 0.25(0) = 0.333$.

(c) Gini prefers **Split 2** ($0.333 < 0.375$) because it produces a *pure* node (region B is all positive). Node purity is valuable even when it does not change the majority-vote error rate: a pure node needs no further splitting and yields confident predictions. Entropy behaves the same way.

Solution — Exercise 8.2 (4/4)

Solution

(d) Classification error is insensitive to node purity: it ignores improvements that do not flip the majority class. Gini and cross-entropy do reward purity, so they are used to *grow* the tree; classification error is adequate when judging (or pruning toward) the final tree's accuracy.

Common mistake

Averaging the two regions' impurities without their size weights — with unequal regions (Split 2) the unweighted average is misleading.

Exercise 8.3 — Cost-complexity pruning [Concept]

Exercise (≈5 min) — try it before the solution

Cost-complexity pruning selects a subtree $T \subset T_0$ minimising

$$\sum_{m=1}^{|T|} \sum_{i:\, x_i \in R_m} (y_i - \hat{y}_{R_m})^2 \;+\; \alpha\, |T|.$$

1. What do $|T|$ and α represent?
2. What subtree results when $\alpha = 0$, and when $\alpha \to \infty$?
3. Why grow a large tree and then prune, rather than stop splitting early?
4. How is α chosen?

Live demo — §3 of the notebook

Run the §3 cell of `chapter_08_lab.ipynb`: ten-fold CV picks $\alpha \approx 0.013$, yet the pruned tree's test MSE (4.62) is *worse* than the depth-4 tree's 3.612 — CV pruned a tree that `max_depth=4` had already regularised.

Solution — Exercise 8.3 (1/2)

Solution

(a) The two ingredients. $|T|$ is the number of terminal nodes (leaves) — the natural measure of tree complexity, since each leaf is one fitted constant. $\alpha \geq 0$ is a tuning parameter: the *price paid per leaf*. The criterion trades training fit (first term) against complexity ($\alpha|T|$), exactly as the lasso trades RSS against $\lambda \sum |\beta_j|$.

(b) The two extremes. $\alpha = 0$: leaves cost nothing, and every extra split can only lower the RSS term, so the full grown tree T_0 minimises the criterion. $\alpha \to \infty$: the penalty dominates, so the best subtree keeps as few leaves as possible — the *root alone*, a single node predicting the overall mean $\bar{y}$.

Interpretation: sliding α from 0 to ∞ walks through a family of subtrees from most to least complex — the tree analogue of a regularisation path.

Solution — Exercise 8.3 (2/2)

Solution

(c) Why grow-then-prune beats early stopping. Early stopping is short-sighted: a split that looks worthless now (tiny RSS reduction) may *enable* a very valuable split just below it — e.g. when two predictors matter only in combination, the first split shows little gain and a stopping rule would quit before the second, decisive split. Growing a large tree first and then pruning keeps such pairs, and cost-complexity pruning conveniently produces a *nested sequence* of subtrees indexed by α.

(d) Choosing α. By K-fold cross-validation: for a grid of α values, find the corresponding best subtree on each training fold, estimate its CV error, pick the α minimising CV error, then refit that subtree on the full data.

Common mistake

Selecting α on the *training* criterion — the RSS term always favours the biggest tree, so training data alone would pick $\alpha = 0$. Only held-out error can choose the penalty.

Extended Exercise 8.1 — Grow a regression tree by hand [Math]

Extended exercise (≈15 min) — try it before the solution

Grow a regression tree on this data set (predictor x, response y):

x	1	2	3	4	5	6
y	3	4	12	13	20	21

1. Give the root prediction and its RSS.
2. Evaluate every candidate split (midpoints of consecutive x) and pick the one minimising total RSS.
3. Split the resulting right child once more.
4. State the final regions, their predictions, and the total RSS reduction.

Solution — Extended Exercise 8.1 (1/6)

Solution

(a) Root. Grand mean $\bar{y} = \frac{3+4+12+13+20+21}{6} = \frac{73}{6} \approx 12.17$, and

$$\mathrm{RSS}_{\text{root}} = \sum y_i^2 - 6\bar{y}^2 = 1179 - \frac{73^2}{6} \approx 290.83.$$

Solution — Extended Exercise 8.1 (2/6)

Solution

(b) Candidate splits. Predict each region by its mean; total RSS $= \mathrm{RSS}_L + \mathrm{RSS}_R$:

split	left | right mean	total RSS
$x < 1.5$	3.0 | 14.0	190.0
$x < 2.5$	3.5 | 16.5	65.5
$x < 3.5$	6.33 | 18.0	86.7
$x < 4.5$	8.0 | 20.5	82.5
$x < 5.5$	10.4 | 21.0	197.2

The split $x < 2.5$ minimises RSS.

Solution — Extended Exercise 8.1 (3/6)

Solution

First split $x < 2.5$. Left $\{3, 4\}$ (mean 3.5, RSS 0.5); right $\{12, 13, 20, 21\}$ (mean 16.5, RSS 65). The RSS drops from 290.83 to 65.5.

Solution — Extended Exercise 8.1 (4/6)

Solution

(c) Split the right child $\{x \geq 2.5\} = \{12, 13, 20, 21\}$ at $x = 3, 4, 5, 6$:

split	left | right mean	child RSS
$x < 3.5$	12.0 | 18.0	38.0
$x < 4.5$	12.5 | 20.5	1.0
$x < 5.5$	15.0 | 21.0	38.0

The split $x < 4.5$ wins, cutting the child's RSS from 65 to 1.0.

Common mistake

Evaluating the second split on *all six* points. Recursive splitting works within one node at a time — only the right child's four points $\{12, 13, 20, 21\}$ enter this computation.

Solution — Extended Exercise 8.1 (5/6)

Solution

(d) Final tree (three leaves).

region	rule	$\hat{y}$
R_1	$x < 2.5$	3.5
R_2	$2.5 \leq x < 4.5$	12.5
R_3	$x \geq 4.5$	20.5

Final total RSS $= 0.5 + 0.5 + 0.5 = 1.5$.

Solution — Extended Exercise 8.1 (6/6)

Solution

Total RSS reduction.

$$290.83 \xrightarrow{\text{split 1}} 65.5 \xrightarrow{\text{split 2}} 1.5,$$

a reduction of $290.83 - 1.5 = 289.33$ (225.33 from the first split, 64.0 from the second). The tree explains almost all variability because the data fall into three tight clusters.

Why stop here: each leaf's remaining RSS is only 0.5; further splits would chase noise and cost-complexity pruning (any $\alpha > 0.5$) would remove them again.

Outline

1 Why this chapter matters

2 Trees

3 Bagging / RF

4 Boosting

5 Python Lab

6 Summary

Bagging: variance reduction by averaging

Algorithm

1. Draw B bootstrap samples of size n.
2. Fit a deep tree to each sample.
3. Average B predictions (regression) or majority-vote (classification).

Free benefits

- Variance reduction via averaging — bias unchanged.
- OOB error: each training point appears in $\sim 63\,\%$ of the bootstrap samples; the held-out $37\,\%$ act as built-in CV.
- No tuning required.

How much does averaging actually buy?

Each tree has variance σ^2; because they are grown on overlapping bootstrap samples, any two are correlated, $\mathrm{Cor}(\hat{f}_b, \hat{f}_{b'}) = \rho$. Expanding the variance of a sum (Ch. 0: B variance terms and $B(B-1)$ covariance terms) gives

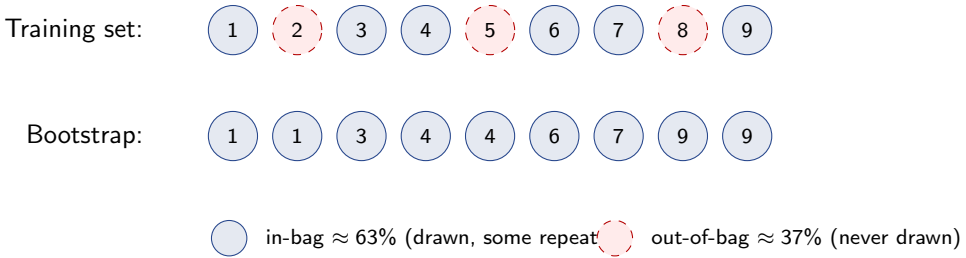
The averaging gain

$$\mathrm{Var}\Big(\tfrac{1}{B}\sum_{b=1}^{B}\hat{f}_b\Big) = \frac{1}{B^2}\left[B\sigma^2 + B(B-1)\rho\sigma^2\right] = \underbrace{\rho\sigma^2}_{\text{floor}} + \underbrace{\frac{1-\rho}{B}\,\sigma^2}_{\to 0}.$$

Two levers, and only one is free

More trees ($B \uparrow$) kills the second term — that is why B is never a tuning parameter and more trees never hurt. But the first term does not move: no matter how many trees you average, the variance stops at $\rho\sigma^2$. Independent trees ($\rho = 0$) would leave $\sigma^2/B \to 0$; identical trees ($\rho = 1$) gain nothing at all. **To go below the floor you must lower** ρ — which is precisely what a random forest does next.

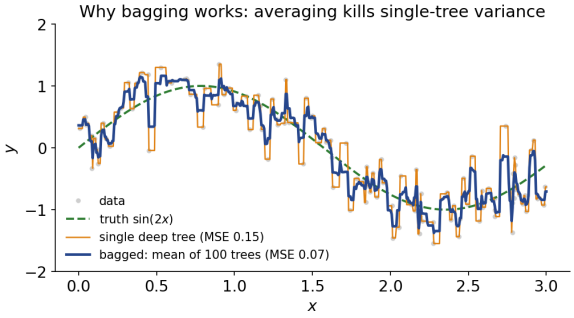
Out-of-bag: the leftover third is a free test set



Takeaway

A bootstrap draw of n points with replacement repeats some (in-bag) and misses others: $\left(1-\frac{1}{n}\right)^n \to e^{-1} \approx 0.37$. Scoring each point with only the trees that did *not* see it gives the OOB error — built-in cross-validation, for free.

Why averaging helps: one deep tree vs. a bagged hundred



Takeaway

On simulated $y = \sin(2x) + \varepsilon$ a single deep tree is a jagged step function (MSE vs. truth 0.15); averaging 100 bootstrap trees smooths it to MSE 0.07 — bagging averages away single-tree variance while the bias stays put.

Random forests: the de-correlation trick

Bagging averages *correlated* trees because the same strong predictor tends to dominate the top split.

Key idea

At each split, consider only m randomly chosen predictors out of p.

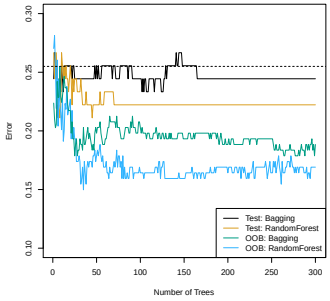
Default values of m

- Classification: $m = \sqrt{p}$.
- Regression: $m = p/3$.

Takeaway

Forcing different trees to use different predictors lowers their pairwise correlation ρ — and so lowers the variance floor $\rho\sigma^2$ that bagging alone cannot get past. With $m = p$ random forests reduce to bagging.

Random forest: error vs. number of trees



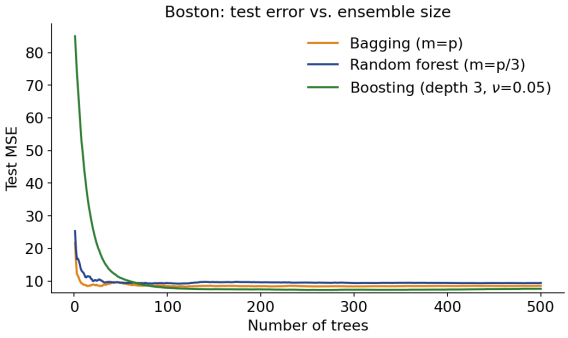
Takeaway

Test error falls then flattens after a few hundred trees. More trees never hurt — unlike boosting, you cannot "overfit" a forest by growing it larger, only hit diminishing returns.

History: bagging (1996) and random forests (2001) are both Leo Breiman's — the same Breiman as in CART.

Source: Figure 8.8 — ISLP, James et al. (2023).

Test error vs. ensemble size: bagging, forest, boosting



Takeaway

All three ensembles beat a single tree. Bagging and the forest fall fast then flatten; slow-learning boosting starts higher but reaches the lowest test MSE.

Variable importance in random forests

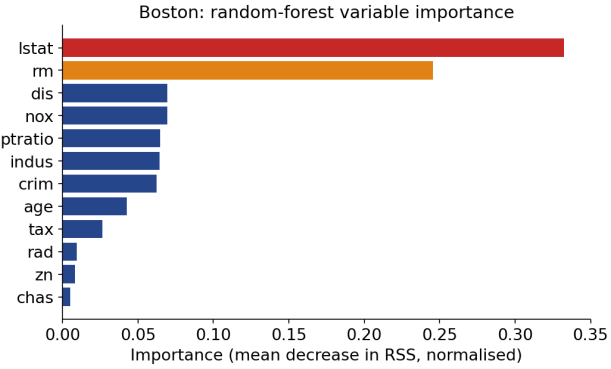
Two notions

- **Impurity importance**: total decrease in node impurity due to splits on X_j, summed across trees.
- **Permutation importance**: shuffle X_j in OOB samples and measure the increase in error.

Caveat

Correlated predictors share importance — you may need to interpret pairs together. Impurity importance is also biased towards high-cardinality variables (customer ID, postcode), so prefer permutation importance when the ranking will be quoted.

Variable importance on the Boston data



Takeaway

Two predictors dominate: `lstat` (% lower-status population) and `rm` (average rooms). Housing value is driven mostly by neighbourhood status and dwelling size; the rest contribute little.

Mini-check: trees and ensembles

Quick self-assessment

1. Why is a single tree typically high-variance?
2. Why does feature subsampling at each split improve random forests over plain bagging?
3. Can you "overfit" a random forest by growing too many trees?

Answers

1. Greedy splits, no shrinkage, perfectly fit training data. 2. Decorrelates the trees, so averaging is more effective. 3. No — adding more trees never hurts, you just hit diminishing returns.

Exercise 8.4 — Bagging and OOB error [Concept]

Exercise (≈5 min) — try it before the solution

1. If each tree has variance σ^2, what is the variance of the average of B trees when they are independent? When they have pairwise correlation ρ?
2. What fraction of observations are out-of-bag (OOB) for a given bootstrap tree? Derive it.
3. What does the OOB error estimate, and why is it almost free?

Live demo — §4 of the notebook

In `chapter_08_lab.ipynb` §4, fit the 500-tree forest with `oob_score=True`: OOB $R^2 = 0.629$ appears without touching the test set (test MSE 2.312), and the next cell draws the importance plot.

Solution — Exercise 8.4 (1/3)

Solution

(a) Variance of the average. For the mean of B trees, expand the variance of a sum (B variance terms, $B(B-1)$ covariance terms $\rho\sigma^2$):

$$\mathrm{Var}\Big(\tfrac{1}{B}\sum_b \hat{f}_b\Big) = \frac{1}{B^2}\Big[B\sigma^2 + B(B-1)\rho\sigma^2\Big] = \rho\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

Independent trees ($\rho = 0$): $\mathrm{Var} = \sigma^2/B \to 0$ — averaging could remove *all* variance. *Correlated trees:*

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \xrightarrow{B\to\infty} \rho\sigma^2.$$

Interpretation: bagging drives the second term to zero for free (more trees), but the floor $\rho\sigma^2$ remains — the correlation between trees caps how much variance averaging can remove. This is exactly the lever random forests attack.

Solution — Exercise 8.4 (2/3)

Solution

(b) OOB fraction. A bootstrap sample makes n independent draws *with replacement*; each draw misses observation i with probability $1 - \frac{1}{n}$, so

$$\Pr(i \text{ never drawn}) = \left(1 - \tfrac{1}{n}\right)^n \xrightarrow{n\to\infty} e^{-1} \approx 0.368$$

(already 0.366 at $n = 100$). So about $1/3$ of observations are OOB and $2/3$ in-bag for each tree.

Solution — Exercise 8.4 (3/3)

Solution

(c) OOB error. Each observation is OOB for roughly $B/3$ of the trees; predicting it using *only those* trees gives an honest held-out prediction. Aggregating over all n observations yields the OOB error, which approximates the test / CV error — and it comes free with a single fit, since it reuses trees that were grown anyway.

Common mistake

Scoring OOB points with *all* B trees. Trees that saw the point in-bag have memorised it, so including them makes the error estimate optimistically low.

Exercise 8.5 — Random forests and decorrelation [Concept/Math]

Exercise (≈5 min) — try it before the solution

1. How does a random forest differ from bagging? What is m?
2. Give the usual choice of m for classification and for regression with p predictors.
3. Using the variance formula $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, explain why restricting to m predictors helps.
4. With $p = 16$, what is m for classification and for regression?

Solution — Exercise 8.5 (1/2)

Solution

(a) The difference. A random forest is bagging plus a twist: at *each split* only a random subset of m of the p predictors is considered as candidates. The subset is redrawn *at every split*, not once per tree, so a single tree can still use all predictors — just never all at the same decision. Bagging is the special case $m = p$.

(b) Default values. Classification: $m \approx \sqrt{p}$. Regression: $m \approx p/3$. These are rules of thumb — m is a tuning parameter worth checking by OOB error or CV.

Interpretation: m is the decorrelation dial: $m = p$ recovers bagging, small m forces diverse trees.

Solution — Exercise 8.5 (2/2)

Solution

(c) Why restricting to m predictors helps. If one or two predictors are very strong, in bagging almost every bootstrap tree splits on them at the top, so the trees look alike — ρ is large and the limiting variance $\rho\sigma^2$ stays high, no matter how big B is. With $m < p$ a split ignores each given predictor with probability $(p - m)/p$ — e.g. $m = 4$ of $p = 16$ hides the dominant predictor at $12/16 = 75\,\%$ of splits — so trees are forced to differ. That lowers ρ, hence the floor $\rho\sigma^2$, and averaging becomes far more effective.

(d) With $p = 16$: classification $m = \sqrt{16} = 4$; regression $m = 16/3 \approx 5$ (round 5.33 down to an integer).

Common mistake

Expecting small m to reduce *bias*. Restricting candidates slightly *raises* each tree's bias; the payoff is purely in decorrelation, i.e. variance reduction of the average.

Outline

1 Why this chapter matters

2 Trees

3 Bagging / RF

4 Boosting

5 Python Lab

6 Summary

Boosting: idea

Boosting builds a sequence of weak trees, each fitted to the residuals of the previous fit. Slow, careful adaptation.

Gradient-boosting algorithm

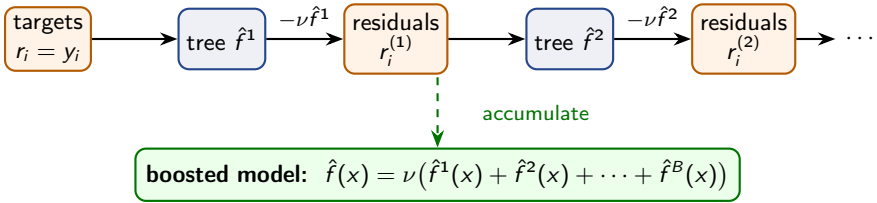
1. Initialise $\hat{f}(x) = 0$, $r_i = y_i$.
2. For $b = 1, \ldots, B$:
   1. Fit a tree $\hat{f}^b$ with d splits to (x_i, r_i).
   2. Update $\hat{f} \leftarrow \hat{f} + \nu \hat{f}^b$.
   3. Update residuals $r_i \leftarrow r_i - \nu \hat{f}^b(x_i)$.

Takeaway

Nothing here fits the response directly after the first step: every tree after it fits the *residual* left by the ones before, and ν decides how much of each correction is kept.

History: AdaBoost (Freund & Schapire, 1997) made boosting practical; Friedman (2001) recast it as gradient boosting, XGBoost's ancestor.

Boosting: each tree fits what the last one missed



Reading the update

Fit a small tree to the current *residuals* r_i (what the model still gets wrong), add only a fraction ν of it ($0<\nu\ll1$), then shrink the residuals. Small ν = slow, cautious learning; the depth d caps the interaction order.

Boosting tuning parameters

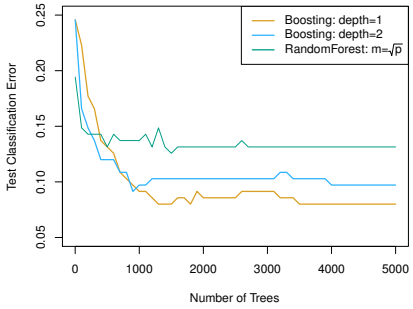
The trio

- B — number of trees. Too large $\Rightarrow$ overfitting.
- ν — learning rate (typically 0.01 or 0.001).
- d — tree depth (often $d = 1$ or 2 suffices).

Trade-off

Smaller ν usually wins but requires larger B. The product $\nu \cdot B$ controls effective fitting effort. Choose all three by CV — but on time-ordered data, hold out *later* periods rather than random folds (Chapter 5), or the future leaks into training and flatters every one of the three.

Boosting vs. random forest: head-to-head



Takeaway

With enough trees and a small learning rate, boosting often edges out the random forest on test error — but it must be tuned carefully, whereas the forest is strong out of the box.

Source: Figure 8.11 — ISLP, James et al. (2023).

Modern boosting libraries

Industrial implementations dominate Kaggle on tabular data.

The big three

`XGBoost`, `LightGBM`, `CatBoost`.

Beyond textbook gradient boosting

Feature subsampling, second-order gradients, ℓ_1 / ℓ_2 regularisation on leaf weights, GPU support, native categorical handling, histogram-based splits.

Takeaway

Native handling of missing values and categorical variables removes most of the feature engineering, and histogram splits make millions of rows routine — which is why these three, not `sklearn`'s textbook implementation, are what actually ships.

Exercise 8.6 — Boosting tuning parameters [Concept]

Exercise (≈5 min) — try it before the solution

Gradient boosting with regression trees has three tuning parameters.

1. Name them and state what each controls.
2. What does "slow learning" mean and why does it help?
3. How do B and λ interact, and what is the danger of a large B?
4. What does d control? What model do we get when $d = 1$?

Live demo — §5 and §7 of the notebook

Run §5 of `chapter_08_lab.ipynb` (B=2000, λ=0.01, d=3 $\rightarrow$ test MSE 2.195 on Carseats), then the §7 table: forest 2.312 vs. boosting 2.195 — both cut the single tree's 3.612 by more than a third.

Solution — Exercise 8.6 (1/2)

Solution

(a) The three parameters. B = number of trees (how many boosting rounds); λ = shrinkage / learning rate (typically 0.01 or 0.001) — the fraction of each new tree actually added; d = number of splits per tree (interaction depth / tree size).

(b) Slow learning. Each new tree is fitted to the current *residuals* — what the ensemble still gets wrong — but only λ times its prediction is added:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x), \qquad r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

With small λ no single tree can dominate; the ensemble creeps toward the signal in many tiny, mutually correcting steps rather than a few large ones. This shrinkage reduces overfitting and improves generalisation — the same logic as ridge/lasso shrinkage, applied to trees.

Solution — Exercise 8.6 (2/2)

Solution

(c) How B and λ interact. A smaller λ requires a larger B to reach the same total fitting effort (roughly, effort $\propto \lambda \cdot B$) — they trade off. The danger: unlike random forests, boosting keeps re-fitting the residuals, so with B too large it starts fitting *noise* — it *can* overfit. Choose B by cross-validation or early stopping on a validation set.

(d) The role of d. d caps how many variables can interact along one tree's path, i.e. the *interaction order*. $d = 1$ gives stumps: each tree involves a single predictor, so the boosted model is purely *additive* — a tree-built cousin of a GAM. Larger d allows higher-order interactions among predictors.

Common mistake

Tuning B by training error — it decreases monotonically in B, so it never signals when to stop; only validation/CV error turns upward.

Outline

1 Why this chapter matters

2 Trees

3 Bagging / RF

4 Boosting

5 Python Lab

6 Summary

Python lab — run it live in the notebook

Companion notebook: `chapter_08_lab.ipynb`

The code in this section is meant to be **demonstrated live**. Open the companion Jupyter notebook and run it cell by cell, section by section:

- §2, *Single regression tree*, and §3, *Pruning via cost-complexity* — on the Carseats data loaded in §1, *The Carseats data*;
- §4, *Random forest*, and §5, *Gradient boosting* — with the *Optional: XGBoost* subsection that closes §5;
- §6, *Why Gini and not classification error?* — Exercise 8.2 re-done in code — and §7, *The four models side by side*.

Data loads via the `ISLP` package or the bundled CSV files; §8, *Exercises*, closes the notebook.

Decision tree on Carseats

```
from sklearn.tree import DecisionTreeRegressor, plot_tree # fit and draw
from sklearn.model_selection import train_test_split # honest test error
from ISLP import load_data # the book's data loader

Carseats = load_data("Carseats") # 400 stores, Sales in 1000s
X = pd.get_dummies(Carseats.drop(columns="Sales"), drop_first=True) # factors -> 0/1
y = Carseats["Sales"] # response: unit sales per store

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0) # seeded 70/30
tree = DecisionTreeRegressor(max_depth=4).fit(Xtr, ytr) # depth cap = the regulariser
plot_tree(tree, feature_names=X.columns, filled=True, fontsize=7) # read rules top-down
```

How to read this

`max_depth=4` is the only regulariser here: splitting stops after four levels, so the tree stays readable instead of growing one leaf per store. Each leaf predicts the mean `Sales` of its training cases — test MSE 3.612. The alternative, `ccp_alpha` $= \alpha$, charges α per leaf and prunes a deep tree back: CV picks $\alpha \approx 0.013$ and scores a *worse* 4.62.

Random forest

```
from sklearn.ensemble import RandomForestRegressor # B bootstrap trees
import numpy as np

rf = RandomForestRegressor(n_estimators=500, max_features="sqrt", # sqrt(p) per split
                           random_state=0, oob_score=True).fit(Xtr, ytr) # keep OOB score
print("OOB R^2:", rf.oob_score_) # 0.629: validation without a test set
print("Test MSE:", ((yte - rf.predict(Xte))**2).mean()) # 2.312 vs 3.612 one tree

imp = pd.Series(rf.feature_importances_, index=X.columns) # total RSS drop per variable
imp.sort_values().plot.barh() # which predictors the forest leans on
```

How to read this

`max_features="sqrt"` is what makes this a *forest* rather than bagging: each split may use only $\sqrt{p}$ of the p predictors, so no single strong variable heads all 500 trees — less correlated trees average better. `oob_score=True` scores each store with the $\approx$37 % of trees that never drew it, so $R^2 = 0.629$ costs no test data. Test MSE 2.312 beats the single tree's 3.612.

Gradient boosting and XGBoost

```
from sklearn.ensemble import GradientBoostingRegressor # trees in sequence
gb = GradientBoostingRegressor(n_estimators=2000, learning_rate=0.01, # B=2000, step 0.01
                               max_depth=3, random_state=0).fit(Xtr, ytr) # d=3, weak
print("Test MSE:", ((yte - gb.predict(Xte))**2).mean()) # 2.195, best of the four

# Optional: XGBoost, the production-grade implementation
import xgboost as xgb # same idea, much faster
bst = xgb.XGBRegressor(n_estimators=2000, learning_rate=0.05, # bigger step, same B
                       max_depth=4, subsample=0.8).fit(Xtr, ytr) # 80% of rows per tree
```

How to read this

Boosting is sequential: each of the $B = 2000$ trees fits what the ensemble still gets wrong, and only `learning_rate` $\lambda = 0.01$ of it is added. Small λ learns cautiously and so needs large B: the product λB is the effective fitting effort, so lowering one means raising the other. `max_depth=3` keeps each tree a weak learner. Test MSE 2.195, just ahead of the forest's 2.312.

Exercise 8.7 — Random forest & boosting on Boston [Python]

Exercise (≈5 min) — try it before the solution

Using `Boston.csv` (target `medv`):

1. Fit a random forest and a gradient-boosting regressor; report test MSE.
2. Read off the most important features from the random forest.
3. Interpret the result.

Run this live

The worked solution sits in `chapter_08_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Exercise 8.7 — Random forest & boosting on Boston [Python]*.

Solution — Exercise 8.7 (1/3)

Solution

```
import pandas as pd # CSV in, DataFrame out
from sklearn.ensemble import (RandomForestRegressor, # the two ensembles
                              GradientBoostingRegressor) # to compare
from sklearn.model_selection import train_test_split # one fixed split
from sklearn.metrics import mean_squared_error # the scoring rule

Boston = pd.read_csv('../ALL CSV FILES - 2nd Edition/Boston.csv') # 506 tracts
Boston = Boston.drop(columns=[c for c in ['Unnamed: 0'] # stray index column
                              if c in Boston.columns]) # only if present
X = Boston.drop(columns='medv'); y = Boston['medv'] # target: median value
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, # 70/30 split
                                      random_state=0) # same split each run
```

Set-up: we hold out 30 % as a test set; fixing `random_state` makes the split (and every number below) reproducible.

Solution — Exercise 8.7 (2/3)

Solution

```
rf = RandomForestRegressor(n_estimators=500, max_features='sqrt', # sqrt(p) per split
                           random_state=0).fit(Xtr, ytr) # train only; test held back
print('RF test MSE:', mean_squared_error(yte, rf.predict(Xte))) # ~19.6 here

gb = GradientBoostingRegressor(n_estimators=1000, learning_rate=0.01, # small steps
                               max_depth=3, random_state=0).fit(Xtr, ytr) # weak learners
print('GBM test MSE:', mean_squared_error(yte, gb.predict(Xte))) # ~13.5: beats the RF

imp = pd.Series(rf.feature_importances_, # RSS drop credited per variable
                index=X.columns).sort_values(ascending=False) # biggest first
print(imp.head()) # lstat and rm lead
```

What it prints (this seed): RF test MSE $\approx$ 19.6, boosting $\approx$ 13.5 — versus $\approx$ 27 for a single unpruned tree on the same split. The importance list is headed by `lstat` ($\approx$ 0.28) and `rm` ($\approx$ 0.27).

Solution — Exercise 8.7 (3/3)

Solution

Interpretation. Both ensembles reach a test MSE well below a single tree (typically around 10–15 for Boston). On this particular split the $m = \sqrt{p}$ forest lands higher (≈ 19.6) while boosting reaches ≈ 13.5: Boston has a few dominant predictors, so restricting each split to $\sqrt{12} \approx 3$ candidates costs accuracy here — Extended Exercise 8.3 shows that a larger m closes the gap.

The importance ranking is dominated by `lstat` (% lower-status population) and `rm` (average number of rooms): housing value depends most on socio-economic status and dwelling size. Importances measure the total RSS reduction attributable to each variable summed across all trees, normalised to sum to 1.

Common mistake

Comparing models fitted or evaluated on *different* splits — always fix one train/test split (or use CV) before declaring a winner, and remember that single-split MSEs carry noticeable seed-to-seed noise.

Extended Exercise 8.3 — Bagging vs. forest vs. boosting [Python]

Extended exercise (≈15 min) — try it before the solution

Using `Boston.csv` (target `medv`), fit and compare three ensembles against a single tree:

1. **Bagging** (a forest that considers all p predictors at each split);
2. a **random forest**, tuning $m =$ features tried per split;
3. **gradient boosting**, tuning $(B, \lambda, d) =$ (trees, learning rate, depth).

Report the test MSE of each, and read off the random-forest variable-importance ranking. Which method wins, and which predictors matter?

Run this live

The worked solution sits in `chapter_08_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Extended Exercise 8.3 — Bagging vs. forest vs. boosting [Python]* — all four fits on one split (seed 1), with m and (B, λ, d) tuned by CV *inside* the training set.

Solution — Extended Exercise 8.3 (1/6)

Solution

Load the data; fit a single tree and bagging (`max_features=p` = bagging).

```
import pandas as pd # CSV loading
from sklearn.model_selection import train_test_split # one split, reused
from sklearn.tree import DecisionTreeRegressor # the baseline
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error as mse # scoring rule
B = pd.read_csv('../ALL CSV FILES - 2nd Edition/Boston.csv') # 506 tracts
B = B.drop(columns=[c for c in ['Unnamed: 0', ''] if c in B.columns]) # tidy
X, y = B.drop(columns='medv'), B['medv']; p = X.shape[1] # p=12 predictors
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.3, random_state=1) # fixed
tree = DecisionTreeRegressor(random_state=1).fit(Xtr, ytr) # unpruned baseline
bag = RandomForestRegressor(n_estimators=500, max_features=p, # m=p -> bagging
                            random_state=1).fit(Xtr, ytr) # all p vars per split
print('tree', round(mse(yte, tree.predict(Xte)), 2), # 26.7 against 8.6
      'bag', round(mse(yte, bag.predict(Xte)), 2)) # averaging kills variance
```

Solution — Extended Exercise 8.3 (2/6)

Solution

What it prints (seed 1): tree ≈ 26.7, bagging ≈ 8.6 — averaging 500 bootstrap trees alone cuts the single tree's test error to a third.

Solution — Extended Exercise 8.3 (3/6)

Solution

Tune the forest's m and boosting's (B, λ, d); read importances.

```
for m in ['sqrt', 4, 6]: # vars tried per split
    rf = RandomForestRegressor(n_estimators=500, max_features=m, # the m dial
                               random_state=1).fit(Xtr, ytr) # only m varies
    print('RF m=', m, round(mse(yte, rf.predict(Xte)), 2)) # 9.81, 9.43, 8.73

for lr, Bn, d in [(0.1, 1000, 2), (0.01, 2000, 3)]: # big step/few vs small/many
    gb = GradientBoostingRegressor(learning_rate=lr, n_estimators=Bn, # lambda*B
                                   max_depth=d, random_state=1).fit(Xtr, ytr) # depth d
    print('GBM', lr, Bn, d, round(mse(yte, gb.predict(Xte)), 2)) # 10.24 vs 7.34

imp = pd.Series(rf.feature_importances_, # from the last (m=6) forest
                index=X.columns).sort_values(ascending=False) # biggest first
print(imp.round(3).head()) # lstat and rm dominate
```

What it prints (seed 1): RF $m{=}\sqrt{p}$: 9.81, $m{=}4$: 9.43, $m{=}6$: 8.73; GBM $(0.1, 1000, 2)$: 10.24, $(0.01, 2000, 3)$: 7.34 — slow learning with more trees wins.

Solution — Extended Exercise 8.3 (4/6)

Solution

Typical test MSE (with `random_state=1`; exact figures vary):

Solution — Extended Exercise 8.3 (5/6)

Solution

single tree	≈ 27
bagging ($m = p$)	≈ 8.6
random forest ($m = \sqrt{p} \;\to\; 6$)	$\approx 9.8 \;\to\; 8.7$
boosting (λ=0.01, B=2000, d=3)	≈ 7.3

Solution — Extended Exercise 8.3 (6/6)

Solution

Ranking: every ensemble slashes the single tree's error 3–4-fold; well-tuned *boosting* edges out bagging and the forest. Because Boston has few, strong predictors, a larger m helps, so bagging is competitive with the forest here. **Importance:** `lstat` (≈ 0.40) and `rm` (≈ 0.27) dominate, then `dis`, `nox` — housing value is driven mostly by neighbourhood socio-economic status and dwelling size. Boosting wins on accuracy but needs tuning; the forest is a strong low-effort default.

Common mistake

Reading importances as causal effects. They measure how much the ensemble *uses* a variable, and correlated predictors share the credit — `lstat`'s 0.40 partly stands in for the whole socio-economic cluster.

Outline

1 Why this chapter matters

2 Trees

3 Bagging / RF

4 Boosting

5 Python Lab

6 Summary

Chapter 8 in one slide

What we accomplished

The dominant family of methods on tabular data: trees and their ensembles.

- Recursive partitioning, pruning, classification trees.
- Bagging averages many trees to cut variance.
- Random forests decorrelate trees by random feature subsetting.
- Boosting fits sequential weak learners to residuals.
- BART: a Bayesian sum of small trees.

Key formulas at a glance

Regression tree minimise $\sum_m \sum_{i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2$; predict region mean.

Gini $G = \sum_k \hat{p}_{mk}(1 - \hat{p}_{mk})$; **entropy** $D = -\sum_k \hat{p}_{mk} \log \hat{p}_{mk}$.

Pruning minimise $\sum_{m=1}^{|T|} \mathrm{RSS}_m + \alpha |T|$ (cost complexity, $\alpha \geq 0$).

Bagging $\hat{f}_{\mathrm{bag}}(x) = \dfrac{1}{B} \sum_{b=1}^{B} \hat{f}^{*b}(x)$; OOB error $\approx$ test error.

Averaging gain $\mathrm{Var}\left(\frac{1}{B}\sum_b \hat{f}_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \xrightarrow{B\to\infty} \rho\sigma^2$ (each $\mathrm{Var} = \sigma^2$, pairwise corr. ρ).

Random forest as bagging but try $m \approx \sqrt{p}$ (class.) or $p/3$ (regr.) predictors per split.

Boosting $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$; tune B, λ, d (slow learning).

Vocabulary checklist

Term	Meaning
Recursive binary splitting	Greedy node-by-node tree growth
Cost-complexity pruning	Penalise tree size by $\alpha\|T\|$
Gini / cross-entropy	Impurity for classification splits
Bagging	Average B bootstrap-trained trees
OOB error	Out-of-bag prediction error
Random forest	Bagging + random feature subsetting
Boosting	Sequential additive trees on residuals
Learning rate ν	Shrinkage applied to each boosting step
BART	Bayesian sum of trees
Variable importance	Total impurity / accuracy decrease per predictor

Decision rules of thumb

- Single tree: only as a baseline or for max interpretability.
- Random forest: an excellent default with very little tuning.
- Boosting: typically best accuracy, tune ν, depth, B jointly.
- Use OOB error in random forests — it's free CV.
- Permutation importance is more honest than impurity importance.
- Trees handle missing data and mixed types out of the box.

Common pitfalls

Don't

1. Use very deep boosted trees with high ν — they overfit.
2. Trust importance rankings under heavy collinearity.
3. Compare a tuned RF to an untuned linear model and conclude RF always wins.
4. Forget that trees fit step-functions — they cannot extrapolate.
5. Skip validation: OOB error is free for a forest, but boosting's B, λ and d still need CV or early stopping.

Self-check questions

Each question names the slide that answers it — a wrong answer tells you where to read.

1. Why is recursive binary splitting greedy? When does this hurt? (p. 14)
2. Why use cross-entropy or Gini, not the misclassification rate, to grow classification trees? (p. 17)
3. Explain why random forests reduce variance more than plain bagging. (pp. 41 and 44)
4. Walk through the gradient-boosting algorithm in pseudocode. (p. 58)
5. In what sense is BART a Bayesian boosting algorithm? (appendix, p. 100)
6. Rank baseline methods (logistic, RF, gradient boosting) for tabular data with $p = 100$, $n = 1000$. (pp. 61 and 86)

Eight things to remember

1. Trees split predictor space into rectangles.
2. Greedy recursive binary splitting; cost-complexity pruning.
3. Single trees are interpretable but high variance.
4. Bagging: variance reduction by averaging.
5. Random forests: bagging + feature subsampling.
6. Boosting: sequential fit to residuals.
7. BART: Bayesian sum of small trees.
8. Tree ensembles dominate tabular ML.

What's next

This was the last taught chapter — from here the semester turns to revision and the final exam.

- **Module A9 — Deep Learning** (ISLP Ch. 10): MLPs, convolutional networks, backpropagation, SGD, regularisation. Self-study, with its own deck and lab; the final mock exam draws on it.
- **Module A7 — Multiple Testing** (ISLP Ch. 13): also assigned as self-study; Short Exam E and the final papers carry a problem from it.
- The remaining modules (A1–A6, A8) are optional depth — SVMs, survival analysis, unsupervised learning, and more.

Appendix — what is in here, and why

Nothing in the appendix is needed to follow the chapter: each slide is a formal derivation, a longer worked exercise, or a side topic. Read it when you want the full story, or when a later chapter sends you back here.

Contents of the appendix

- **ISLP Figures 8.1 and 8.2 — the Hitters tree and its partition** — the book's rendering; the main deck teaches from the redrawn flowchart version.
- **The partition picture, drawn a second time** — a schematic duplicate of the Hitters partition already shown in the main deck.
- **Impurity measures and pruning (Extended Exercise 8.2)** — Gini, entropy and cost-complexity pruning worked through in full. Exercises 8.2 and 8.3 cover both in the main deck.
- **BART** — Bayesian additive regression trees — a fourth ensemble, listed for completeness. Random forests and boosting are the two you will be asked about.

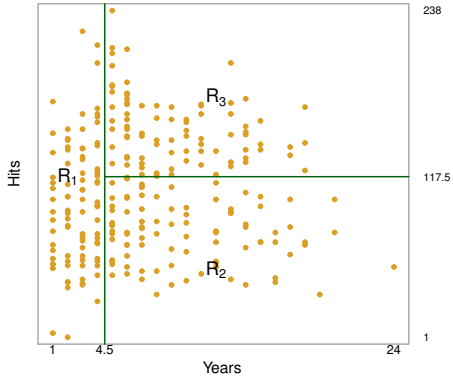
Hitters: a baseball-salary tree



Years and Hits split the salary space into three regions; each leaf predicts the regional mean log-salary.

Source: Figure 8.1 — ISLP, James et al. (2023).

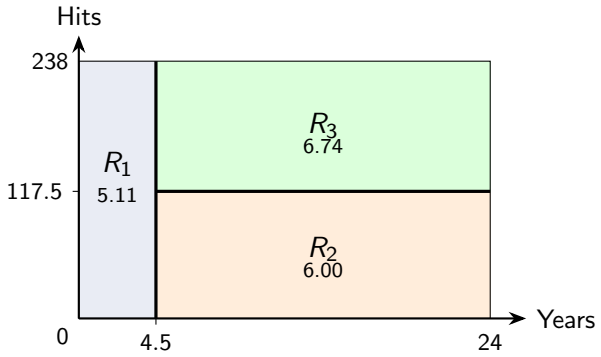
Hitters: how the tree partitions the data



Each leaf is a rectangle in (Years, Hits).

Source: Figure 8.2 — ISLP, James et al. (2023).

The same tree as a partition of predictor space



Takeaway

R_1 holds inexperienced players; among experienced players, more hits (R_3) predicts higher salary than fewer (R_2).

Extended Exercise 8.2 — Impurity measures and pruning [Math]

Extended exercise (≈15 min) — try it before the solution

A binary-classification node holds 200 cases: 100 "Yes" and 100 "No". Two candidate splits give:

- **Split A:** $A_1 = (80\text{ Yes}, 20\text{ No})$ and $A_2 = (20\text{ Yes}, 80\text{ No})$.
- **Split B:** $B_1 = (100\text{ Yes}, 40\text{ No})$ and $B_2 = (0\text{ Yes}, 60\text{ No})$.

1. For each split compute the weighted Gini index, cross-entropy and classification error.
2. Which split wins under each criterion? Explain the disagreement.
3. A pruned-tree sequence has subtrees with (leaves $|T|$, training errors R) $= (4, 0), (3, 4), (2, 10), (1, 40)$. For $\alpha \in \{0, 5, 20, 50\}$ find the subtree minimising $R + \alpha|T|$.

Solution — Extended Exercise 8.2 (1/4)

Solution

For a node with "Yes"-fraction $\hat{p}$:

$$G = 2\hat{p}(1-\hat{p}), \quad D = -\hat{p}\ln\hat{p} - (1-\hat{p})\ln(1-\hat{p}), \quad E = 1 - \max(\hat{p}, 1-\hat{p}),$$

and children are combined by their share of the 200 cases.

Split A (A_1, A_2 each 100 cases, $\hat{p} = 0.8$ and 0.2 — symmetric):

$$G = 2(0.8)(0.2) = 0.32, \quad D = -0.8\ln 0.8 - 0.2\ln 0.2 = 0.500, \quad E = 0.20.$$

Both children give identical values and equal weights $\frac{100}{200} = 0.5$, so $G_A = 0.32$, $D_A = 0.500$, $E_A = 0.20$.

Note: we use the natural log for D; switching to $\log_2$ rescales every entropy by the same factor and never changes which split wins.

Solution — Extended Exercise 8.2 (2/4)

Solution

Split B. B_1: $\hat{p} = 100/140 = 0.714$, weight 0.7; B_2 pure ($\hat{p} = 0$), weight 0.3.

$$G_B = 0.7\,[2(0.714)(0.286)] + 0.3\,(0) = 0.7(0.408) = 0.286,$$
$$D_B = 0.7\,[-0.714 \ln 0.714 - 0.286 \ln 0.286] + 0 = 0.7(0.598) = 0.419,$$
$$E_B = 0.7\,(1 - 0.714) + 0.3\,(0) = 0.7(0.286) = 0.20.$$

(2) Who wins? Classification error ties ($E_A = E_B = 0.20$) and cannot separate the splits. Gini ($0.286 < 0.32$) and entropy ($0.419 < 0.500$) both prefer **Split B**, because it carves out a *pure* node B_2. Purity is rewarded by Gini/entropy even when majority-vote error is unchanged — which is why we *grow* trees with Gini or entropy, not error.

Solution — Extended Exercise 8.2 (3/4)

Solution

(3) Cost-complexity pruning. Minimise $R_\alpha(T) = R(T) + \alpha|T|$ over the nested subtrees:

	$(4,0)$	$(3,4)$	$(2,10)$	$(1,40)$
α=0	0	4	10	40
α=5	20	19	20	45
α=20	80	64	50	60
α=50	200	154	110	90

Winners: α=0 $\to$ 4 leaves, α=5 $\to$ 3 leaves, α=20 $\to$ 2 leaves, α=50 $\to$ the root. As α grows the optimal subtree shrinks (breakpoints at $\alpha = 4, 6, 30$) — the nested "weakest-link" sequence that cross-validation then searches over.

Solution — Extended Exercise 8.2 (4/4)

Solution

Sample entry: for $\alpha = 5$ and the 3-leaf tree, $R_\alpha = R + \alpha|T| = 4 + 5 \cdot 3 = 19$ — the smallest value in its row, so 3 leaves win at that price.

Where the breakpoints come from. Two neighbouring subtrees swap places where their R_α values tie: pruning one leaf pays when the per-leaf price exceeds the training error it costs, $\alpha > \Delta R / \Delta|T|$. Here the trade-offs are $(4-0)/1 = 4$ leaves $4 \to 3$, $(10-4)/1 = 6$ for $3 \to 2$ and $(40-10)/1 = 30$ for $2 \to 1$ — the breakpoints $\alpha = 4, 6, 30$ of the table. This "price per leaf" is exactly the weakest-link ordering that software uses to build the pruning sequence.

Common mistake: choosing α by training error — $\alpha = 0$ (the full tree) always wins there, by construction. α is chosen by cross-validating over the pruning sequence.

BART: Bayesian Additive Regression Trees

A sum of K small trees, $\hat{f}(x) = \sum_{k=1}^{K} \hat{f}_k(x)$, updated over B sweeps — but unlike boosting, a sweep does not fit a fresh tree:

One update of tree k in sweep b (Algorithm 8.3)

1. Form the **partial residual** — what the *other* $K-1$ trees currently leave unexplained: $r_i = y_i - \sum_{k'<k} \hat{f}_{k'}^{\,b}(x_i) - \sum_{k'>k} \hat{f}_{k'}^{\,b-1}(x_i)$.
2. Randomly **perturb** the previous version of tree k — grow or prune a branch, or change its leaf values — favouring perturbations that fit r_i better.

Why perturb instead of refit — and what you get

Refitting greedily would chase the residual hard and overfit; small random perturbations keep every tree weak, so BART regularises the way slow learning does in boosting. The final prediction *averages* the sweeps after a burn-in of L: $\hat{f}(x) = \frac{1}{B-L} \sum_{b=L+1}^{B} \hat{f}^{\,b}(x)$ — bagging's trick, applied across iterations, and the spread of those sweeps gives uncertainty bands for free. Defaults ($K = 200$, $B = 1000$, $L = 100$) work well out of the box. Implementations: `ISLP.bart`, `PyMC`.

Source: Section 8.2.4, Algorithm 8.3 — ISLP, James et al. (2023).